

SQL Interview Questions — 5+ Years Experience

1. Query Writing & Filtering

1. Write a query to find the **second-highest salary** from an employee table.
2. Find the **Nth highest salary** without using `LIMIT/OFFSET`.
3. Find employees whose salary is greater than the **average salary of their department**.
4. Find employees who joined in the **last 30 days**.
5. Find records created **today, yesterday, and the previous 7 days**.
6. Find duplicate records based on `email`.
7. Delete duplicate records while keeping the **latest record**.
8. Find employees who have **never received a promotion**.
9. Find customers who have made **more than 3 transactions**.
10. Find the **top 5 highest-paid employees in each department**.

2. Joins

11. Explain the difference between `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, and `FULL OUTER JOIN`.
12. Find customers who **have never placed an order**.
13. Find customers who placed an order but **have no payment record**.
14. Find employees whose salary is greater than their manager's salary.
15. Find records existing in Table A but **not in Table B**.
16. Write a query to identify **orphan records**.
17. Explain the difference between `NOT IN`, `NOT EXISTS`, and `LEFT JOIN ... IS NULL`.
18. What happens when you join two tables where one side contains duplicate keys?
19. Write a query to find customers who purchased **every product in a category**.
20. Find customers who purchased **Product A but never Product B**.

3. Window Functions

21. Difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`.
22. Find the **top 3 salaries per department**.
23. Find the **second transaction of every customer**.
24. Find the **latest record for each customer**.

25. Calculate a **running total of sales**.
26. Calculate a **7-day moving average**.
27. Find the difference between the current transaction and the previous transaction.
28. Find customers whose transaction amount is greater than their **previous transaction**.
29. Find the first and last transaction for every customer.
30. Find consecutive login days for users.

4. Date & Time Problems

31. Find the number of transactions for each day.
32. Find customers active on **3 consecutive days**.
33. Find the first transaction date for every customer.
34. Find customers whose **first transaction occurred in 2026**.
35. Calculate month-over-month revenue growth.
36. Find the highest revenue month.
37. Find the number of customers acquired each month.
38. Find customers who were active last month but not this month.
39. Calculate the difference between `created_date` and `modified_date`.
40. Generate a report containing **every date**, including dates with zero transactions.

5. Aggregation

41. Find departments having more than 10 employees.
42. Find the department with the highest average salary.
43. Find the percentage contribution of each department to total salary.
44. Find products contributing to **80% of total revenue**.
45. Find the average transaction amount per customer.
46. Find customers whose total spending is greater than the overall customer average.
47. Find the maximum transaction amount for every customer.
48. Find the number of unique customers per month.
49. Find the percentage of successful vs failed transactions.
50. Find the cancellation rate for each product.

6. CTE & Recursive SQL

51. What is a CTE? When would you use one?
52. Difference between a CTE and a subquery.

- 53. Can a CTE improve query performance?
- 54. Write a recursive CTE to generate numbers from **1 to 100**.
- 55. Generate an organizational hierarchy using a recursive CTE.
- 56. Find all managers under a particular senior manager.
- 57. Find the complete parent-child hierarchy of an employee.
- 58. Use a CTE to identify duplicate customers.
- 59. Use multiple CTEs to calculate a multi-step business metric.
- 60. Explain when a CTE can make a query slower.

7. SQL Performance & Optimization

- 61. A query is taking **30 seconds**. How would you troubleshoot it?
- 62. What is an index?
- 63. What is a composite index?
- 64. Explain **B-tree indexes**.
- 65. When can an index actually make performance worse?
- 66. Why might PostgreSQL not use an index even when one exists?
- 67. Explain **EXPLAIN** and **EXPLAIN ANALYZE**.
- 68. Difference between **Index Scan, Sequential Scan, Bitmap Heap Scan**.
- 69. How would you optimize a query containing multiple joins?
- 70. How would you optimize a query containing **GROUP BY** on millions of rows?
- 71. Why should you avoid **SELECT *** in production queries?
- 72. How does applying a function to an indexed column affect performance?
- 73. Why can **LIKE '%abc%'** be slow?
- 74. How would you optimize a query filtering by a date column?
- 75. What is table partitioning and when would you use it?

8. Advanced PostgreSQL

- 76. Difference between **VARCHAR**, **TEXT**, and **CHAR**.
- 77. Difference between **DELETE**, **TRUNCATE**, and **DROP**.
- 78. Explain PostgreSQL **JSON** vs **JSONB**.
- 79. How do you extract values from a JSON/JSONB column?
- 80. How do you convert a JSON array into rows?
- 81. How do you aggregate rows into a JSON array?
- 82. Explain **LATERAL JOIN**.

- 83. Difference between `COALESCE`, `NULLIF`, and `CASE`.
- 84. How does PostgreSQL handle `NULL`?
- 85. Explain transaction isolation levels.
- 86. What is a deadlock and how can you troubleshoot it?
- 87. What are `VACUUM` and `ANALYZE`?
- 88. What is table bloat?
- 89. What is an UPSERT in PostgreSQL?
- 90. Explain `INSERT ... ON CONFLICT`.

9. Real-World Data Engineering Scenarios

- 91. A source table receives **10 million records every day**. How would you design an incremental SQL load?
- 92. How would you identify records that were **inserted or modified since the last successful load**?
- 93. Design a SQL query for **incremental loading using `modified_date`**.
- 94. What problems can occur when using only `modified_date` as a watermark?
- 95. How would you handle two records having the **same modified timestamp**?
- 96. How would you implement **SCD Type 2** using SQL?
- 97. How would you identify changed records between source and target?
- 98. How would you remove duplicates from a **100-million-row table** efficiently?
- 99. A query works on 1 million rows but becomes extremely slow on 500 million rows. How would you investigate it?
- 100. Your SQL query produces different results after a new join is added. How would you identify whether the join is causing **record multiplication**?

★ Most Important 15 for Your 5+ Year Interview

If you're preparing specifically for **Senior Data Engineer / Data Engineer interviews**, I'd prioritize these:

Priority	Question
🔥 1	Top N records per group using Window Functions
🔥 2	Delete duplicates while keeping latest record
🔥 3	ROW_NUMBER vs RANK vs DENSE_RANK
🔥 4	NOT EXISTS vs NOT IN vs LEFT JOIN
🔥 5	Running total & moving average
🔥 6	Consecutive dates
🔥 7	Month-over-month growth
🔥 8	SCD Type 2
🔥 9	Incremental loading
🔥 10	Query optimization using EXPLAIN ANALYZE
🔥 11	Index selection and composite indexes
🔥 12	Handling duplicate keys after joins
🔥 13	JSON/JSONB manipulation
🔥 14	Recursive CTE
🔥 15	Designing SQL for hundreds of millions of records

For your profile, I would **not spend much time on basic questions like “What is SELECT?”**. The interview value is much higher in **scenario-based SQL + PostgreSQL optimization + Data Engineering SQL problems**.